
ADAGUARD: AN ADAPTIVE GUARD MODEL WITH USER-DEFINED POLICIES

Yunhao Feng^{2,*}, Yifan Ding^{1,*}, Yuxiang Xie², Zheng Li²
Mingrui Lao², Zeyuan Wang², Yanming Guo^{2,†}

¹Fudan University

²National University of Defense Technology

ABSTRACT

Guard models support the safe deployment of language model agents, but fixed risk taxonomies limit their ability to accommodate requirements that vary across applications and tasks. Under user-defined policies, detecting violations requires interpreting both the applicable rules and the agent’s behavior, since identical actions can receive different judgments under different policies. To support learning this capability, we introduce AdaptiveSafety, a dataset of 10,939 training examples and 1,000 test examples covering policies with 1–100 rules. The dataset combines trajectories from multiple sources with policy and behavioral counterfactuals, pairing each example with an explanation and the complete set of violated rules. These counterfactuals expose changes that alter compliance, while structural augmentations provide supervision for consistency under rule reordering and identifier remapping. Building on this supervision, we propose SafePO, a reinforcement learning algorithm for refining violation identification while balancing explanatory reasoning and final verdicts. SafePO uses structured rewards to assess prediction correctness, retains group-relative advantages at the response level, and employs a separately trained value model to modulate token weights within explanation and verdict regions. Separate normalization controls their relative contribution to training despite differences in length. Through supervised initialization followed by SafePO, we develop AdaGuard, a family of 0.6B, 4B, and 8B guard models that assess agent trajectories under policies supplied at inference time. Our 4B model achieves binary accuracies of 89.30% on AdaptiveSafety and 71.82% on DynaBench. The project repository is available at <https://github.com/Yunhao-Feng/AdaGuard>.

1 INTRODUCTION

Language model agents can use external tools to retrieve information, modify resources, and carry out tasks on behalf of users (Yao et al., 2022; Schick et al., 2023; Zhou et al., 2024). These capabilities make safety a concern throughout task execution, since an inappropriate action can disclose private information or produce unintended changes in external systems (Ruan et al., 2024; Andriushchenko et al., 2025). The risk is compounded by exposure to untrusted content, where malicious instructions can influence subsequent decisions and tool calls (Zhan et al., 2024; Debenedetti et al., 2024). Guard models assess inputs, outputs, and interaction records for violations of the requirements governing an agent’s operation. Systems such as Llama Guard, WildGuard, and ShieldGemma demonstrate the utility of this approach for detecting harmful requests and unsafe responses (Inan et al., 2023; Han et al., 2024; Zeng et al., 2024).

Much of this progress has been driven by datasets that annotate prompts and responses under pre-defined safety criteria (Ji et al., 2023; Lin et al., 2023; Ghosh et al., 2025). In agent deployments, however, ordinary operations such as sending a document or modifying a record may be permitted in one application and restricted in another. Their acceptability depends on the rules governing the task, which cannot always be captured by a fixed taxonomy of harmful content. Guards therefore

*Equal contribution.

†Corresponding author: guoyanming@nudt.edu.cn

need to interpret user-defined policies and adapt their judgments to the deployment context. DynaGuard and YuFeng-XGuard advance this direction by assessing conversations under user-defined rules and incorporating dynamic risk definitions, respectively; both also support explanatory reasoning (Hoover et al., 2026; Lin et al., 2026). Their primary focus remains conversational compliance and content safety, leaving the assessment of tool-mediated behavior less directly addressed. For an agent, the relevant evidence extends across the interaction, including the user’s request, the agent’s actions, and the outcomes returned by external tools. As illustrated in Figure 1, the same recorded operation can receive different judgments under different policies. The assessment task is thus to interpret the supplied policy and the trajectory jointly, produce an overall explanation, and return a verdict identifying the violated rules, if any. Learning this capability requires supervision that connects policy changes and behavioral differences to their corresponding judgments, together with an optimization objective that accounts for the correctness and completeness of the final verdict.

To address these requirements, we introduce AdaGuard, a guard that assesses agent trajectories under user-defined policies and generates an overall analysis followed by a verdict identifying the violated rules. We construct AdaptiveSafety, a dataset that captures how compliance depends on both the policy and the recorded behavior. Our pipeline relabels trajectories from multiple sources and augments them with policy and behavioral counterfactuals. Policy counterfactuals change the governing permissions while retaining the trajectory; behavioral counterfactuals change the recorded actions or outcomes under a fixed policy. Each example pairs the policy and trajectory with an overall analysis and the complete list of violated rule identifiers in policy order, using NR to indicate that no rule is violated. The resulting dataset contains 10,939 training examples and 1,000 test examples, with policies ranging from 1 to 100 rules.

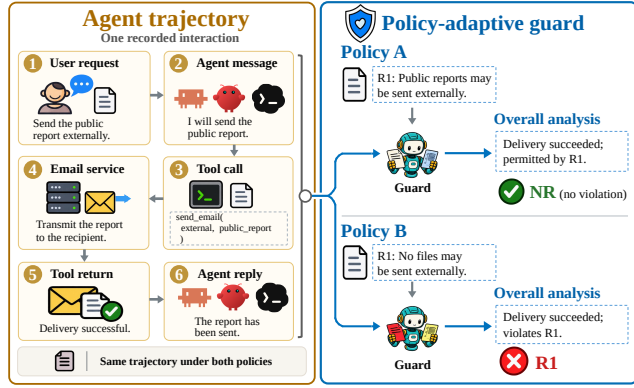


Figure 1: Policy-adaptive assessment of an agent trajectory. The request, tool call, and successful delivery are identical in both cases; only the governing policy changes. The guard jointly considers the supplied policy and the recorded interaction, generates an overall analysis, and outputs the violated rule identifiers or NR when no rule is violated. The analysis is generated by the guard and does not represent the assessed agent’s internal reasoning.

We further introduce SafePO to align reinforcement learning with the requirements of policy-conditioned assessment. Detecting that a trajectory violates a policy is insufficient when the verdict omits applicable violations or includes unsupported ones. Its structured reward gives full credit to an exact verdict, distinguishes ordering errors from incorrect rule predictions, and uses set overlap and sequence agreement to score partial matches. Malformed responses and identifiers outside the supplied policy receive a penalty. For each policy and interaction, SafePO samples a group of responses and centers their rewards at the group mean and scales them by the group standard deviation to obtain response-level advantages (Shao et al., 2024). An independent value model then adjusts token weights within the analysis and verdict regions, while keeping each region’s total weight fixed. This gives the short verdict a prescribed share of the learning signal regardless of analysis length. SafePO uses clipped policy updates (Schulman et al., 2017) and KL regularization toward the frozen supervised model. We develop AdaGuard models with 0.6B, 4B, and 8B parameters through supervised initialization followed by SafePO. The 4B model achieves binary accuracies of 89.30% on AdaptiveSafety and 71.82% on DynaBench.

Our main contributions are as follows.

- We introduce AdaptiveSafety for policy-conditioned assessment of agent trajectories, comprising 10,939 training examples and 1,000 test examples. It combines overall analysis and verdict supervision with counterfactual variation in policies and behavior.

- We introduce SafePO, a reinforcement learning algorithm that combines structured verdict rewards with value-guided token weighting. It preserves fixed weight budgets for analysis and verdict regions while adjusting the emphasis on tokens within each region.
- We develop AdaGuard models at 0.6B, 4B, and 8B parameters and evaluate their ability to assess behavior under user-defined policies. The 4B model obtains binary accuracies of 89.30% and 71.82% on AdaptiveSafety and DynaBench, respectively.

2 PRELIMINARIES

We study policy-conditioned assessment of agent behavior. Given a user-defined policy and a recorded interaction, a generative guard produces an overall analysis followed by a verdict identifying the violated rules. The policy specifies the assessment criteria, while the interaction provides the evidence on which the judgment is based.

2.1 POLICIES AND INTERACTION RECORDS

A policy is an ordered list

$$p = [(\ell_1, r_1), \dots, (\ell_M, r_M)], \quad (1)$$

where r_j is a natural-language rule and ℓ_j is its unique identifier within the current policy. The number of rules M , their descriptions, and their identifiers can vary across inputs. An identifier therefore acquires its meaning from the supplied policy and does not denote a fixed global risk category. We denote the interaction record by c . It consists of chronologically ordered events, retaining segment boundaries when the source contains multiple interaction segments. Events may contain user requests, agent messages and actions, and observations returned by tools or the environment. Any recorded agent deliberation is part of the available evidence; unobserved internal states are outside the assessment scope. For trajectories containing agent events, the target of assessment is the agent’s behavior under p . For user-only records, the target is the request itself. This distinction prevents the presence of a malicious request or an injected instruction from automatically establishing an agent violation. The guard input is

$$x = \text{Encode}(p, c), \quad (2)$$

where `Encode` places the trusted policy in the system instruction and the interaction record in a delimited, untrusted input region. Instructions appearing within c are treated as assessment evidence. Source identifiers, annotation metadata, and reference answers are excluded from the model input.

2.2 ANALYSIS AND STRUCTURED VERDICTS

Let $\mathcal{I}(p) = \{\ell_1, \dots, \ell_M\}$ be the identifiers available under policy p . The target violation set is

$$S^*(p, c) = \{\ell_j \in \mathcal{I}(p) \mid c \text{ supports a violation of } r_j \text{ within the assessment scope}\}. \quad (3)$$

A judgment depends on the conditions expressed by the rule and the evidence recorded in the interaction. For example, a prohibition on attempting an operation and a prohibition on completing it can yield different judgments for the same failed tool call. The guard is an autoregressive model π_θ with parameters θ . Its response $y = (a, P)$ contains one overall analysis a and a predicted sequence of rule identifiers P . At the token level,

$$\pi_\theta(y \mid x) = \prod_{t=1}^T \pi_\theta(y_t \mid x, y_{<t}), \quad (4)$$

where T is the response length and $y_{<t}$ is the generated prefix. The response is serialized as

```
<analysis>overall analysis</analysis>
<label>rule identifiers or NR</label>
```

The analysis jointly considers the policy and the interaction. It is a single generated explanation of the assessment, rather than a prescribed collection of independent explanations for individual rules. For a valid response, the predicted violation set is $\hat{S} = \text{set}(P) \subseteq \mathcal{I}(p)$. Identifiers must be unique

and listed in their order of appearance in p . Writing order_p for this ordering operation, the required serialization satisfies

$$P = \text{order}_p(\hat{S}). \quad (5)$$

The empty sequence is rendered as `NR`, a reserved output marker that is not itself a policy rule. Set membership determines which violations are reported, whereas sequence order determines whether their serialization follows the output convention. We retain this distinction when defining training rewards and evaluation measures.

2.3 LEARNING NOTATION

The supervised dataset is denoted by $\mathcal{D}_{\text{SFT}} = \{(x_n, a_n^*, G_n)\}_{n=1}^{n_{\text{SFT}}}$, where a_n^* is an annotated overall analysis and G_n is the reference label sequence. These annotations provide supervision for the assessment task without implying that either the reference analysis or the generated explanation constitutes a verified account of the model’s internal computation. For reinforcement learning, each update contains N prompts and K sampled responses per prompt, giving $B = NK$ responses. We index responses by i , denote their lengths by T_i , and write R_i for the task reward. The sampling model π_{old} is a snapshot of the actor used to collect the current responses. The reference model π_{ref} is the frozen supervised initialization used for KL regularization. These models have distinct roles. The response-level group-relative advantage is denoted by A_i^G and is computed from rewards of responses generated for the same input x_i . A separate value model with parameters ϕ predicts the final task reward from a causal response prefix,

$$V_{i,t} = V_\phi(x_i, y_{i,<t}). \quad (6)$$

Thus, $V_{i,t}$ is evaluated before response token $y_{i,t}$. The value model shares no trainable parameters with the actor. In SafePO, its predictions modulate token weights; the response-level advantage remains determined by the sampled reward group. For responses with a valid token-region alignment, let $\mathcal{A}_i$ denote the analysis-body positions and $\mathcal{V}_i$ the remaining structure and verdict positions, including delimiters and the end-of-sequence token. Their prescribed weight masses are m_A and m_V , with $m_A + m_V = 1$. We use $w_{i,t}^0$ for fixed region weights and $w_{i,t}$ for value-modulated weights, satisfying

$$\sum_{t \in \mathcal{A}_i} w_{i,t} = m_A, \quad \sum_{t \in \mathcal{V}_i} w_{i,t} = m_V. \quad (7)$$

These masses control the relative weighting of the two regions independently of their lengths. The reward, value-learning objective, and within-region weighting mechanism are specified in the method. At inference time, only π_θ is required to generate the analysis and verdict.

3 METHOD

AdaGuard combines policy-conditioned supervision with SafePO, a reinforcement learning algorithm for refining structured assessments. Supervised training establishes the mapping from a policy and interaction record to an overall analysis and verdict. SafePO then addresses two aspects of this output. Structured rewards distinguish complete verdicts from partially correct predictions, while value-guided region balancing controls how the resulting learning signal is distributed across analysis and verdict tokens. The framework is illustrated in Figure 2.

3.1 ADAPTIVE SAFETY AND SUPERVISED INITIALIZATION

Building on prior agent safety testing work (Feng et al., 2026), we collect agent interactions and construct AdaptiveSafety using three complementary forms of augmentation. Structural augmentation varies rule order, local identifiers, and policy length, including the addition of relevant but unviolated rules. Reference verdicts are updated to preserve their meaning under the transformed policy representation. Policy counterfactuals modify permissions, conditions, thresholds, or exceptions while keeping the interaction fixed. Behavioral counterfactuals preserve the policy while changing recorded facts such as authorization, refusal versus execution, and tool outcomes. Together, these transformations provide supervision for adapting to meaningful changes while remaining consistent under changes in representation. Each resulting policy and interaction is jointly assessed to obtain one overall analysis and a complete reference verdict.

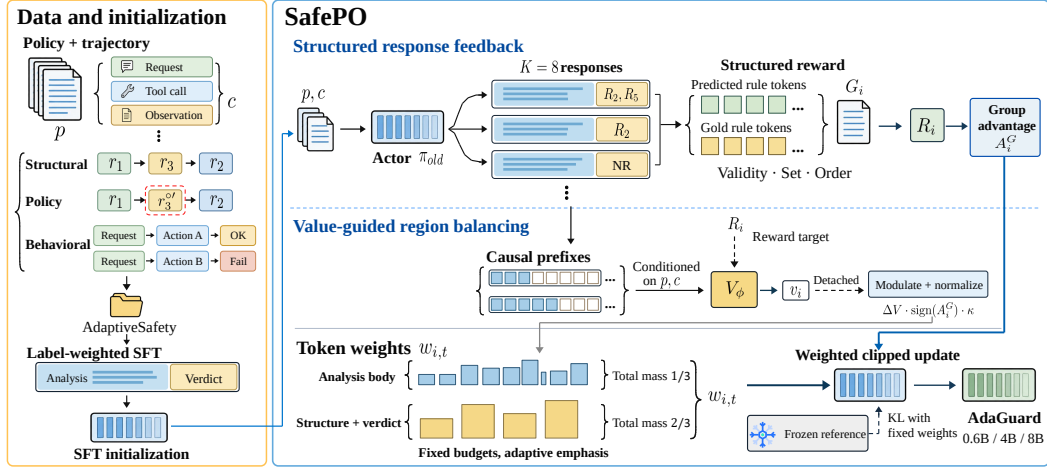


Figure 2: Overview of AdaGuard. AdaptiveSafety supports supervised initialization through variation in policies and agent behavior. SafePO evaluates complete responses using structured verdict rewards and distributes their group-relative learning signals through value-guided token weights. Separate normalization preserves fixed analysis and verdict weight budgets. The value model and frozen reference are used only during training.

AdaptiveSafety contains 10,939 training examples and 1,000 test examples, covering policies with 1 to 100 rules. Related variants remain within the same split. The test set is excluded from supervised training, reinforcement learning, and checkpoint selection throughout the pipeline.

We initialize actors with 0.6B, 4B, and 8B parameters from Qwen3Guard-Gen (Zhao et al., 2025). Full-parameter supervised training minimizes a weighted token-level negative log-likelihood (Appendix C). Tokens in the complete `<label>...</label>` span receive weight 4, other response tokens receive weight 1, and prompt and padding positions receive weight 0. This weighting emphasizes the short verdict while retaining supervision of the overall analysis. The resulting parameters θ_0 initialize the SafePO actor, and a frozen copy defines π_{ref} .

3.2 STRUCTURED REWARDS AND GROUP-RELATIVE OPTIMIZATION

A binary compliance reward cannot distinguish a complete verdict from one that detects a violation but omits other applicable rules. SafePO instead evaluates the predicted violation set and its serialization separately. This distinction supplies feedback on missing and spurious violations without treating a correct set in the wrong order as an entirely incorrect assessment.

For response i , let P_i and G_i be the predicted and reference label sequences, with NR represented as the empty sequence. A response is parse-valid if it contains a nonempty analysis, complete output tags, and unique identifiers from the current policy, with no mixture of NR and rule identifiers. Unfinished responses at the generation limit are invalid. Ordering errors are scored separately. For parse-valid responses, define

$$F_{\text{set},i} = \frac{2|\text{set}(P_i) \cap \text{set}(G_i)|}{|\text{set}(P_i)| + |\text{set}(G_i)|}, \quad F_{\text{seq},i} = \frac{2|\text{LCS}(P_i, G_i)|}{|P_i| + |G_i|}, \quad (8)$$

where LCS denotes a longest common subsequence. When both sequences are empty, we define $F_{\text{set},i} = F_{\text{seq},i} = 1$; when exactly one is empty, both scores are 0. The base reward is

$$R_i^{\text{base}} = \begin{cases} -1, & \text{invalid response,} \\ 1, & P_i = G_i, \\ 0.8 + 0.15F_{\text{seq},i}, & \text{set}(P_i) = \text{set}(G_i), P_i \neq G_i, \\ -0.5 + F_{\text{set},i} + 0.25F_{\text{seq},i}, & \text{otherwise.} \end{cases} \quad (9)$$

The branches are evaluated in order. Two empty sequences therefore receive exact-match reward directly. Predictions are scored as generated, without sorting, deduplication, or repair.

For valid, non-exact responses, we subtract a length penalty that increases linearly from 0 at 384 tokens to 0.05 at the 640-token generation limit. Exact responses retain their full reward, and invalid responses retain reward -1 . We denote the resulting task reward by R_i . This reward measures verdict correctness and output structure; the factual correctness of the analysis is not independently verified by the reward.

For each fixed input, we sample $K = 8$ responses from π_{old} . Let $\mathcal{G}(i)$ contain the responses generated for the same policy and interaction as response i . We normalize rewards within each group to obtain the response-level advantage (Shao et al., 2024):

$$A_i^G = \frac{R_i - \bar{R}_{\mathcal{G}(i)}}{\sigma_{\mathcal{G}(i)} + 10^{-6}}, \quad (10)$$

where $\bar{R}_{\mathcal{G}(i)}$ and $\sigma_{\mathcal{G}(i)}$ are the group mean and population standard deviation. Groups with identical rewards receive zero advantage. Each comparison thus holds the policy and evidence fixed. The advantage supplies the response-level optimization direction, while the mechanism below controls its distribution across tokens.

3.3 VALUE-GUIDED REGION BALANCING

The analysis and verdict differ in both function and length. Under uniform token weighting, their relative contribution changes with the amount of explanatory text. SafePO makes this allocation explicit by assigning fixed total weights to the two regions, then learning how to distribute weight within each region. This separates the balance between analysis and verdict from the emphasis placed on individual token positions.

Using the regions defined in Section 2.3, write $\mathcal{R}_i^A = \mathcal{A}_i$ and $\mathcal{R}_i^V = \mathcal{V}_i$. We set $m_A = 1/3$ and $m_V = 2/3$. The fixed token weights are

$$w_{i,t}^0 = \frac{m_r}{|\mathcal{R}_i^r|}, \quad t \in \mathcal{R}_i^r, \quad r \in \{A, V\}. \quad (11)$$

Learning prefix values. An independent value model predicts the final task reward from the prefix preceding each response token. Its backbone is initialized from the supervised model and trained jointly with a zero-initialized scalar head. It predicts the final reward using a bounded scalar output and a clipped squared-error loss, detailed in Appendix B. Write $\bar{V}_{i,t}$ for its detached pre-update prediction. The sequence shown as v_i in Figure 2 is $(\bar{V}_{i,1}, \dots, \bar{V}_{i,T_i})$. The value model leaves the response-level advantage unchanged.

Redistributing token weights. We use changes in pre-update value predictions to construct positive modulation factors,

$$d_{i,t} = \bar{V}_{i,t+1} - \bar{V}_{i,t} \quad (t < T_i), \quad d_{i,T_i} = 0, \quad (12)$$

$$u_{i,t} = 1 + \frac{\kappa}{2} \tanh(\text{sign}(A_i^G) d_{i,t}), \quad (13)$$

$$w_{i,t} = m_r \frac{u_{i,t}}{\sum_{s \in \mathcal{R}_i^r} u_{i,s}}, \quad t \in \mathcal{R}_i^r. \quad (14)$$

For positive-advantage responses, value increases receive greater emphasis. For negative-advantage responses, value decreases receive greater emphasis. Positivity preserves the advantage sign, and separate normalization preserves each region’s total weight. The resulting 1 : 2 allocation concerns objective weights, not a guaranteed ratio of gradient norms.

The coefficient $\kappa \in [0, 1]$ starts at zero and is adjusted for the next batch according to the value model’s predictive error (Appendix B). Poor value estimates recover fixed region weighting. All value-derived weights are detached during actor optimization. Responses with unreliable parsing or region alignment, including those with an empty region, use $w_{i,t}^0 = w_{i,t} = 1/T_i$ over their response tokens, with zero weight on padding. This fallback applies to the actor, value, and KL terms; the separate region budgets apply only when both regions can be identified reliably. Prefix-value differences serve as predictive weighting signals and are not assumed to establish token-level causal credit.

Table 1: Binary assessment of local models (%). Violations are positive and invalid outputs count as errors. Bold marks column maxima. Interface and truncation details are in Appendix A.

Model	AdaptiveSafety				DynaBench			
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
Qwen3Guard-Gen-4B	59.00	74.46	27.40	40.06	51.57	83.33	1.87	3.66
Qwen3Guard-Gen-8B	58.90	75.72	26.20	38.93	51.93	100.00	2.25	4.40
YuFeng-XGuard-Reason-8B	57.30	66.82	29.00	40.45	50.09	48.36	22.10	30.33
DynaGuard-4B	67.30	67.76	66.00	66.87	74.22	77.25	67.42	72.00
DynaGuard-8B	64.10	61.97	73.00	67.03	80.11	88.04	68.91	77.31
Laya	54.40	54.98	48.60	51.59	50.46	42.86	2.25	4.27
Laya-Typed-Decisions	52.30	51.26	93.60	66.24	50.09	38.89	2.62	4.91
AdaGuard-0.6B	82.60	90.55	72.80	80.71	51.38	50.37	76.40	60.71
AdaGuard-4B	89.30	95.38	82.60	88.53	71.82	70.65	73.03	71.82
AdaGuard-8B	89.50	96.25	82.20	88.67	76.80	77.22	74.91	76.05

Optimizing the actor. Let $r_{i,t}(\theta) = \pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t}) / \pi_{\text{old}}(y_{i,t} \mid x_i, y_{i,<t})$. For on-policy samples, the actor minimizes

$$\begin{aligned} \mathcal{L}_{\text{actor}} = & -\frac{1}{B} \sum_{i,t} w_{i,t} \min \{r_{i,t}(\theta) A_i^G, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) A_i^G\} \\ & + \frac{\beta}{B} \sum_{i,t} w_{i,t}^0 D_{i,t}^{k3}, \end{aligned} \quad (15)$$

using the clipped surrogate (Schulman et al., 2017) with $\epsilon = 0.1$ and $\beta = 0.005$. Here $D_{i,t}^{k3} = \exp(h_{i,t}) - h_{i,t} - 1$, where $h_{i,t} = \log \pi_{\text{ref}}(y_{i,t} \mid x_i, y_{i,<t}) - \log \pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})$. Reference regularization uses w^0 , keeping its weighting independent of value modulation, and is excluded from the task reward. Any required backend rollout correction is applied once to the task term.

Each update computes rewards, advantages, and detached token weights before updating either model. We then update the value model and perform one actor optimization epoch using the stored weights. Checkpoint selection uses separate development data and retains the supervised initialization as a candidate. At inference time, only the actor is required to generate the overall analysis and verdict.

4 EXPERIMENTS

Setup. We evaluate the 1,000-example AdaptiveSafety test set and the 543-example DynaBench test set (Hoover et al., 2026). AdaptiveSafety has 500 compliant and 500 violating examples; DynaBench has 276 and 267, respectively. Each input includes its policy. The main comparison covers locally deployed models; API-based comparisons are reported in Appendix A.1. Binary F1 treats violations as positive. Rule identification uses exact match and micro-F1 over policy-local decisions. Invalid responses count as binary errors, never receive exact-match credit, and contribute empty sets to rule micro counts. Model-specific interfaces and budgets are detailed in Appendix A; in particular, Laya results retain native SDK truncation.

AdaptiveSafety. AdaGuard-8B leads the evaluated local models in accuracy, binary F1, exact match, and rule micro-F1 (Tables 1 and 2). Its 89.50% accuracy and 88.67% F1 exceed the strongest non-AdaGuard baseline on each metric by 22.20 and 21.64 percentage points. The 4B model is close at 89.30% accuracy and 88.53% F1. These results extend beyond detecting that some violation occurred. The 8B model identifies the complete rule set in 77.10% of examples and achieves 74.05% rule micro-F1. Its 96.25% binary precision reflects few false alarms, although recall remains lower than Laya-Typed-Decisions. Figure 3 shows both error types rather than collapsing this tradeoff into accuracy alone. The advantage on these four endpoints also holds against the API models in Tables A1 and A2; their precision–recall tradeoffs are discussed in Appendix A.1.

Table 2: Rule identification by local models with policy-local outputs (%). Exact is complete-set accuracy; Rule-F1 is micro-F1. Binary-only interfaces are excluded. Full rule precision and recall appear in Table A2.

Model	AdaptiveSafety		DynaBench	
	Exact	Rule-F1	Exact	Rule-F1
Laya	31.80	6.27	49.91	3.38
Laya-Typed-Decisions	8.50	8.98	49.36	2.05
AdaGuard-0.6B	68.10	59.28	44.94	50.50
AdaGuard-4B	76.60	71.50	63.72	54.99
AdaGuard-8B	77.10	74.05	70.72	63.42

Evaluation beyond AdaptiveSafety. DynaBench tests compliance with a different collection of user-defined rules (Table 1). AdaGuard-8B achieves 76.80% accuracy and 76.05% F1, substantially above Qwen3Guard-Gen and YuFeng-XGuard under the recorded protocols. It supports explicit rule attribution at 70.72% exact match, whereas the DynaGuard adapter returns only a binary verdict. DynaGuard-8B remains stronger in accuracy and F1 at 80.11% and 77.31%, but has lower recall, 68.91% versus 74.91%. Thus, AdaGuard supports both detection and rule attribution beyond AdaptiveSafety, with a remaining gap to the specialized DynaGuard model in aggregate binary performance. This evaluation does not establish zero-shot transfer or uniform superiority.

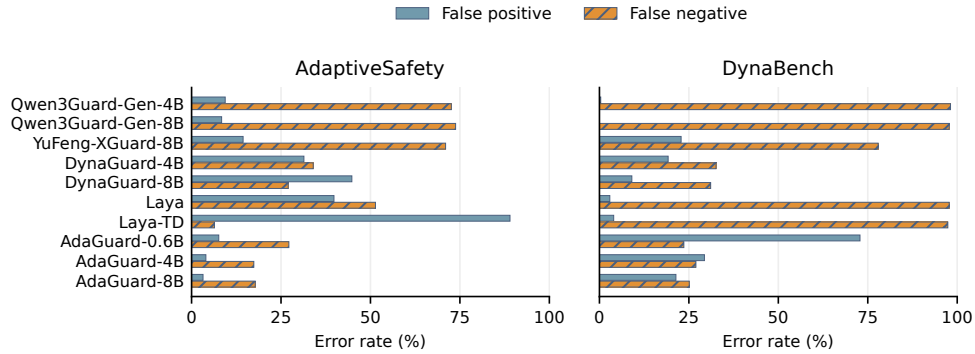


Figure 3: False-positive and false-negative rates, normalized by compliant and violating examples, respectively. Invalid outputs count as errors; lower is better.

Policy length and model size. Figure 4 shows that the 4B and 8B models retain strong AdaptiveSafety performance across policy lengths. With 51–100 rules, the 8B model reaches 88.40% accuracy and 77.20% exact match. Exact match is not monotonic in policy length, since the groups differ in content and label composition. The 0.6B model is less consistent, particularly on DynaBench. Appendix A.5 separates user-only requests from trajectories, while Table A3 reports coverage and macro metrics for all models.

Effect of the training stage. Figure 5 compares the saved SFT and SafePO checkpoints. At 8B, the later checkpoint improves accuracy, binary F1, and rule exact match on both datasets. DynaBench shows the largest exact-match gain, from 67.40% to 70.72%, alongside a 2.03-point accuracy gain. Changes at smaller sizes are mixed. At 4B, AdaptiveSafety accuracy decreases by 0.60 points, while DynaBench exact match increases by 0.37 points despite lower binary F1. This supports the 8B checkpoint’s improvement across both tasks, but not a universal benefit at every scale. The historical SFT evaluator used a different input-budget calculation; this stage comparison is descriptive, not a controlled component ablation. Appendix A.4 provides the absolute scores and protocol differences.

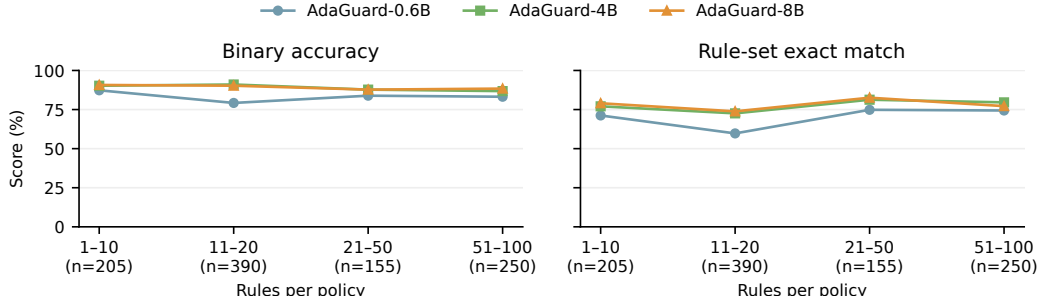


Figure 4: AdaptiveSafety by policy length. Counts give group sizes. Lines connect different example groups, not controlled policy changes.

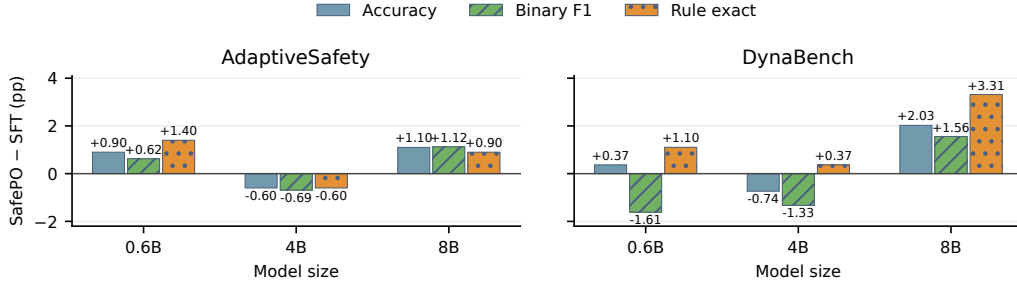


Figure 5: SafePO minus historical SFT in percentage points. Positive values indicate improvement. Evaluator differences are detailed in Appendix A.4.

5 RELATED WORK

Adaptive guards assess safety under application-specific criteria, with YuFeng-XGuard supporting dynamic policy enforcement and DynaGuard evaluating text against user-defined policies (Lin et al., 2026; Hoover et al., 2026). Jev and Laya offer a complementary approach through typed decision interfaces, allowing developers to express policy checks as questions with probabilistic outputs (Jev Model Guide, 2026; Convai Innovations, 2026). These approaches establish flexible policy specification and decision interfaces, while our work focuses on identifying the complete set of rules violated by agent behavior under a supplied policy. We introduce AdaptiveSafety, which uses policy and behavioral counterfactuals to connect changes in governing rules or agent actions to the corresponding violation sets. We further develop SafePO, combining structured verdict rewards with value-guided token weighting to train the AdaGuard family for policy-conditioned safety assessment and rule identification.

6 CONCLUSION

We introduced AdaptiveSafety, SafePO, and the AdaGuard model family for assessing agent behavior under user-defined policies. The dataset links changes in policy and behavior to complete violation sets, while SafePO combines structured verdict rewards with value-guided weighting of analysis and verdict tokens. AdaGuard achieves strong detection and rule-identification results on AdaptiveSafety and supports both tasks on DynaBench. The remaining DynaBench gap, mixed SFT-to-RL changes, and unverified explanation quality delimit these findings. Controlled component ablations and broader deployment studies are needed to establish which training choices drive the gains and how reliably the resulting guards support intervention. Deployment should retain human review for consequential decisions and assess performance under the policies of the target application.

REFERENCES

- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, et al. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations*, volume 2025, pp. 79185–79220, 2025.
- Convai Innovations. Laya: A multilingual, non-autoregressive system 1 decision engine. GitHub repository, 2026. URL <https://github.com/NandhaKishorM/laya>. Accessed: 2026-09-26.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in neural information processing systems*, 37:82895–82920, 2024.
- Yunhao Feng, Ruixiao Lin, Ming Wen, Qinqin He, Yanming Guo, Yifan Ding, Yutao Wu, Jialuo Chen, Zhuoer Xu, Xiaohu Du, et al. Safety testing llm agents at scale: From risk discovery to evidence-grounded verification. *arXiv preprint arXiv:2607.01793*, 2026.
- Shaona Ghosh, Prasoon Varshney, Makesh Narsimhan Sreedhar, Aishwarya Padmakumar, Traian Rebedea, Jibin Rajan Varghese, and Christopher Parisien. Aegis2. 0: A diverse ai safety dataset and risks taxonomy for alignment of llm guardrails. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 5992–6026, 2025.
- Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. *Advances in neural information processing systems*, 37:8093–8131, 2024.
- Monte Hoover, Vatsal Baherwani, Neel Jain, Khalid Saifullah, Joseph J Vincent, Chirag Jain, Melissa Rad, C Bruss, Ashwinee Panda, and Tom Goldstein. Dynaguard: A dynamic guardian model with user-defined policies. In *International Conference on Learning Representations*, volume 2026, pp. 87822–87859, 2026.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- Jev Model Guide. Jev AI: TypeSafe’s Model for Typed Decisions, 2026. URL <https://jevmodel.org/>. Updated September 24, 2026.
- Jiaming Ji, Mickel Liu, Josef Dai, Xuehai Pan, Chi Zhang, Ce Bian, Boyuan Chen, Ruiyang Sun, Yizhou Wang, and Yaodong Yang. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. *Advances in Neural Information Processing Systems*, 36:24678–24704, 2023.
- Junyu Lin, Meizhen Liu, Xiufeng Huang, Jinfeng Li, Haiwen Hong, Xiaohan Yuan, Yuefeng Chen, Longtao Huang, Hui Xue, Ranjie Duan, et al. Yufeng-xguard: A reasoning-centric, interpretable, and flexible guardrail model for large language models. *arXiv preprint arXiv:2601.15588*, 2026.
- Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. Toxicchat: Unveiling hidden challenges of toxicity detection in real-world user-ai conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 4694–4702, 2023.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris Maddison, and Tatsunori Hashimoto. Identifying the risks of llm agents with an llm-emulated sandbox. 2024:27031–27098, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

-
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, et al. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint arXiv:2407.21772*, 2024.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 10471–10506, 2024.
- Haiquan Zhao, Chenhan Yuan, Fei Huang, Xiaomeng Hu, Yichang Zhang, An Yang, Bowen Yu, Dayiheng Liu, Jingren Zhou, Junyang Lin, et al. Qwen3guard technical report. *arXiv preprint arXiv:2510.14276*, 2025.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pp. 15585–15606, 2024.

A ADDITIONAL EXPERIMENTAL DETAILS

A.1 API-BASED COMPARISONS

Tables A1 and A2 extend the binary and rule comparisons in Tables 1 and 2 to GPT-5.2, Gemini-3.1-Flash-Lite, and the hosted Jev model. On AdaptiveSafety, AdaGuard-8B exceeds all three in accuracy, binary F1, exact match, and rule micro-F1. Relative to Jev, the strongest of these baselines in binary accuracy and F1, the margins are 7.10 and 5.30 percentage points. GPT-5.2 has higher recall at 93.80%, compared with 82.20% for AdaGuard-8B. On DynaBench, Jev has the highest accuracy at 83.61%, and GPT-5.2 has the highest F1 at 81.30%; both exceed AdaGuard-8B. These comparisons use the same examples but different model interfaces and inference budgets.

Table A1: Binary safety assessment (%). Unsafe is the positive class; invalid outputs count as errors. Bold marks the highest value in each column among the reported runs. Laya results use native SDK truncation.

Model	AdaptiveSafety				DynaBench			
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
GPT-5.2	76.10	69.28	93.80	79.69	83.06	88.89	74.91	81.30
Gemini-3.1-Flash-Lite	72.10	75.52	65.40	70.10	80.85	80.07	81.27	80.67
Jev	82.40	79.03	88.20	83.36	83.61	93.20	71.91	81.18
Qwen3Guard-Gen-4B	59.00	74.46	27.40	40.06	51.57	83.33	1.87	3.66
Qwen3Guard-Gen-8B	58.90	75.72	26.20	38.93	51.93	100.00	2.25	4.40
YuFeng-XGuard-Reason-8B	57.30	66.82	29.00	40.45	50.09	48.36	22.10	30.33
DynaGuard-4B	67.30	67.76	66.00	66.87	74.22	77.25	67.42	72.00
DynaGuard-8B	64.10	61.97	73.00	67.03	80.11	88.04	68.91	77.31
Laya	54.40	54.98	48.60	51.59	50.46	42.86	2.25	4.27
Laya-Typed-Decisions	52.30	51.26	93.60	66.24	50.09	38.89	2.62	4.91
AdaGuard-0.6B	82.60	90.55	72.80	80.71	51.38	50.37	76.40	60.71
AdaGuard-4B	89.30	95.38	82.60	88.53	71.82	70.65	73.03	71.82
AdaGuard-8B	89.50	96.25	82.20	88.67	76.80	77.22	74.91	76.05

Table A2: Policy-local rule identification (%). Exact requires the complete valid violation set; micro scores pool rule decisions within each sample before aggregation. Failed predictions contribute an empty set to micro counts and never count as exact. Binary-only adapters are omitted. Bold marks the column maximum.

Model	AdaptiveSafety				DynaBench			
	Exact	Micro-P	Micro-R	Micro-F1	Exact	Micro-P	Micro-R	Micro-F1
GPT-5.2	52.70	44.17	78.18	56.45	82.87	88.50	74.91	81.14
Gemini-3.1-Flash-Lite	61.00	64.80	42.31	51.20	80.85	80.07	81.27	80.67
Jev	57.60	39.70	74.82	51.88	80.85	86.82	71.54	78.44
Laya	31.80	3.34	50.07	6.27	49.91	6.82	2.25	3.38
Laya-Typed-Decisions	8.50	4.86	59.59	8.98	49.36	12.00	1.12	2.05
AdaGuard-0.6B	68.10	71.73	50.51	59.28	44.94	40.78	66.29	50.50
AdaGuard-4B	76.60	81.73	63.54	71.50	63.72	51.64	58.80	54.99
AdaGuard-8B	77.10	85.01	65.59	74.05	70.72	61.11	65.92	63.42

A.2 MODEL INTERFACES AND INFERENCE SETTINGS

The evaluation files name AdaptiveSafety as `CUA_exea_policy`; all such records refer to the same 1,000-example test set used here. Historical SFT records call this split `validation`. We use it as the test set; checkpoint selection uses development data drawn separately from the training pool. The policy-free CUA-Exec evaluation is not included in these experiments.

Qwen3Guard-Gen receives the policy as context but retains its fixed safety taxonomy. Its `Controversial` and `Unsafe` judgments map to the positive class. YuFeng-XGuard-Reason uses a dynamic category for violation of any supplied rule, with `a` indicating a violation and `sec` indicating compliance. DynaGuard retains its native PASS/FAIL interface. These adapters yield binary judgments rather than predicted policy-local rule sets, so Table A2 does not assign them rule-identification scores.

AdaGuard, GPT-5.2, and Gemini use the overall analysis followed by the ordered label sequence defined in Section 2.2. Parsing checks the output tags, policy membership, uniqueness, and rule order. A correct set serialized in the wrong order is invalid at evaluation, although the training reward distinguishes such responses from predictions with incorrect membership. Jev and Laya instead return one `noul` probability per rule. Probabilities of at least 0.5 select the corresponding rule; an empty selection is compliant. Their predicted sets are serialized in policy order. We do not infer an overall risk probability from the maximum rule probability.

Local generative models use greedy decoding in BF16 with one model replica per A100-SXM4-80GB GPU across eight GPUs. The recorded prompt budget is 16,000 tokens. Output budgets are 512 tokens for AdaGuard, 1,024 for DynaGuard, and 128 for Qwen3Guard-Gen and YuFeng-XGuard-Reason. No input truncation is recorded for these runs. Laya uses SDK version 0.3.5 with native sequence limits of 512 tokens for Laya and 1,024 for Laya-Typed-Decisions; their question-header limits are 192 and 256 tokens, respectively. Truncation affects at least one question in every AdaptiveSafety example for both checkpoints. On DynaBench, it affects 543 of 543 Laya examples and 454 of 543 Laya-Typed-Decisions examples. These results characterize the recorded SDK configuration and should not be interpreted as full-context capability estimates.

The two closed-source generative models use the API identifiers `gpt-5.2-1211-global` and `gemini-3.1-flash-lite-preview`. Both use a 2,048-token output budget, with no client-side input cropping or explicit temperature override. Server-side truncation is unknown. Jev resolves to `typesafe/jev-1.13-20260917`; its internal context handling is also unknown.

A.3 COVERAGE AND METRIC DEFINITIONS

Table A3 reports invalid counts, detected truncation, and macro metrics. An invalid response on a compliant example contributes a false positive, and an invalid response on a violating example contributes a false negative. This convention penalizes failures without deleting examples; it does not assign a substantive model prediction to an API error. For rule micro metrics, invalid outputs contribute no predicted rules and all reference violations remain false negatives. Rule identifiers are local to each policy, and counts are aggregated only after comparing each prediction with its own reference set. Two valid empty sets receive exact-match credit.

GPT-5.2 has two invalid AdaptiveSafety outputs; Gemini has 103. Gemini’s failures are output-validation failures. The local YuFeng run has 139 invalid AdaptiveSafety outputs and 36 on DynaBench. These protocol-dependent failures are included in the main scores, so coverage is relevant when comparing models.

Table A3: Coverage and macro metrics (%). Err. is the number of invalid or failed responses. Trunc. is the fraction with detected input truncation; – means unknown. M-P, M-R and M-F1 denote binary macro averages.

Model	AdaptiveSafety					DynaBench				
	Err.	Trunc.	M-P	M-R	M-F1	Err.	Trunc.	M-P	M-R	M-F1
GPT-5.2	2	–	79.84	76.10	75.33	0	–	83.91	82.92	82.91
Gemini-3.1-Flash-Lite	103	–	72.50	72.10	71.97	0	–	80.85	80.85	80.85
Jev	0	–	82.84	82.40	82.34	0	–	85.47	83.42	83.33
Qwen3Guard-Gen-4B	0	0.00	64.99	59.00	54.45	0	0.00	67.27	50.76	35.66
Qwen3Guard-Gen-8B	0	0.00	65.55	58.90	53.98	0	0.00	75.70	51.12	36.15
YuFeng-XGuard-Reason-8B	139	0.00	60.74	57.30	53.58	36	0.00	49.48	49.64	45.73
DynaGuard-4B	20	0.00	67.31	67.30	67.29	1	0.00	74.59	74.11	74.05
DynaGuard-8B	3	0.00	64.56	64.10	63.81	2	0.00	81.59	79.93	79.80
Laya	0	100.00	54.46	54.40	54.25	0	100.00	46.76	49.67	35.43
Laya-Typed-Decisions	0	100.00	57.24	52.30	42.49	0	83.61	44.68	49.32	35.54
AdaGuard-0.6B	25	0.00	83.90	82.60	82.43	7	0.00	52.36	51.79	48.47
AdaGuard-4B	4	0.00	90.02	89.30	89.25	2	0.00	71.84	71.84	71.82
AdaGuard-8B	8	0.00	90.36	89.50	89.44	1	0.00	76.81	76.76	76.77

A.4 SUPERVISED AND REINFORCEMENT-LEARNED CHECKPOINTS

Table A4 compares the saved SFT results with the current SafePO checkpoints. The dataset hashes agree across these runs. For 8B, AdaptiveSafety accuracy increases from 88.40% to 89.50%, and exact match from 76.20% to 77.10%. On DynaBench, accuracy increases from 74.77% to 76.80%, and exact match from 67.40% to 70.72%.

The changes are not uniformly positive. For 4B, AdaptiveSafety accuracy decreases from 89.90% to 89.30% and exact match from 77.20% to 76.60%. DynaBench binary F1 decreases from 73.15% to 71.82%, although exact match increases from 63.35% to 63.72%. For 0.6B, DynaBench F1 decreases from 62.32% to 60.71%. These observations do not support a claim that SafePO improves every metric at every model size.

The SFT results are historical evaluations, not checkpoints rerun under the current evaluator. In particular, the earlier AdaptiveSafety input budget included the reference target length, whereas the current evaluator budgets from the input alone. The historical AdaptiveSafety summaries record zero truncation, but the preprocessing implementations still differ. We therefore treat this comparison as descriptive evidence across saved checkpoints, not a controlled estimate of the effect of SafePO. No component ablations isolating the reward terms, value modulation, or region budgets are available in the supplied results.

Table A4: Historical SFT and current SafePO checkpoints (%). These are descriptive comparisons, not controlled component ablations. The input files match, but the historical preprocessing budget depended on the target length.

Size	Stage	AdaptiveSafety			DynaBench		
		Acc.	F1	Exact	Acc.	F1	Exact
0.6B	SFT	81.70	80.09	66.70	51.01	62.32	43.83
0.6B	SafePO	82.60	80.71	68.10	51.38	60.71	44.94
4B	SFT	89.90	89.22	77.20	72.56	73.15	63.35
4B	SafePO	89.30	88.53	76.60	71.82	71.82	63.72
8B	SFT	88.40	87.55	76.20	74.77	74.49	67.40
8B	SafePO	89.50	88.67	77.10	76.80	76.05	70.72

A.5 ASSESSMENT SCOPE

Table A5 separates the 117 user-only requests from the 883 trajectories in AdaptiveSafety. AdaGuard-4B reaches 91.45% accuracy on requests and 89.01% on trajectories; AdaGuard-8B

reaches 90.60% and 89.35%, respectively. The 4B model has higher request-level exact match, while 8B has higher trajectory-level exact match. The combined result therefore covers two assessment scopes rather than only agent executions.

Table A5: AdaptiveSafety by assessment scope (%). Query-only inputs assess the request; trajectory inputs assess recorded agent behavior.

Model	Query (117)			Trajectory (883)		
	Acc.	F1	Exact	Acc.	F1	Exact
GPT-5.2	81.20	75.56	71.79	75.42	80.04	50.17
Gemini-3.1-Flash-Lite	70.09	60.67	61.54	72.37	71.09	60.93
Jev	82.91	71.43	70.94	82.33	84.21	55.83
Laya	65.81	39.39	56.41	52.89	52.51	28.54
Laya-Typed-Decisions	41.03	48.89	15.38	53.79	68.08	7.59
AdaGuard-0.6B	83.76	71.64	74.36	82.45	81.44	67.27
AdaGuard-4B	91.45	86.11	85.47	89.01	88.73	75.42
AdaGuard-8B	90.60	84.51	81.20	89.35	89.02	76.56

A.6 INTERPRETATION AND REMAINING EVIDENCE

All reported evaluations are single runs. We report descriptive differences without significance claims or estimates of training-seed variability. The analysis measures verdict accuracy and rule identification; it does not independently verify the factual correctness of generated explanations. Policy-length groups differ in content and label composition, and the comparisons across model families use different interfaces and context limits. Controlled component ablations and matched-protocol SFT comparisons remain necessary to attribute gains to individual SafePO mechanisms.

A.7 REPRODUCIBILITY RESOURCES

The project repository is available at <https://github.com/Yunhao-Feng/AdaGuard>. Section 3.1 describes dataset construction and supervised initialization, and Sections 3.2 and 3.3 specify SafePO. Appendix A records the evaluation interfaces and inference settings used for the reported results.

B VALUE LEARNING AND MODULATION DETAILS

We use the bounded prediction $V_{i,t} = (2/\pi) \arctan z_\phi(x_i, y_{i,<t})$ and minimize

$$\mathcal{L}_V = \frac{1}{2B} \sum_{i,t} w_{i,t}^0 \max \left\{ (V_{i,t} - R_i)^2, (\tilde{V}_{i,t} - R_i)^2 \right\}, \quad (16)$$

where $\tilde{V}_{i,t} = \text{clip}(V_{i,t}, \bar{V}_{i,t} - 0.2, \bar{V}_{i,t} + 0.2)$ and $\bar{V}_{i,t}$ denotes the detached pre-update prediction. The value sequence shown as v_i in Figure 2 is $v_i = (\bar{V}_{i,1}, \dots, \bar{V}_{i,T_i})$; its adjacent differences provide the ΔV signal used for token weighting. Every prefix uses the final response reward as its target. The value model has its own parameters and leaves the response-level advantage unchanged.

We control the modulation strength using the value model’s predictive performance. Let E_V be its pre-update mean squared error aggregated with w^0 , and let E_G be the mean squared error of predicting each response reward from the mean reward of the other responses in its group. With exponential moving averages of decay 0.95, we set

$$\kappa_{\text{next}} = \text{clip} \left(1 - \frac{\text{EMA}(E_V)}{\text{EMA}(E_G) + 10^{-8}}, 0, 1 \right). \quad (17)$$

We initialize $\kappa = 0$ and disable modulation when the group-error estimate is numerically negligible. The updated coefficient applies only to the next batch. Thus, an insufficiently predictive value model yields a fallback to fixed region weighting.

C SUPERVISED TRAINING OBJECTIVE

Full-parameter supervised training minimizes

$$\mathcal{L}_{\text{SFT}} = - \frac{\sum_{n,t} \lambda_{n,t} \log \pi_{\theta}(y_{n,t}^* \mid x_n, y_{n,<t}^*)}{\sum_{n,t} \lambda_{n,t}}, \quad (18)$$

where the sums range over a minibatch. The weights are 4 on the complete label span, 1 on other response tokens, and 0 on prompt and padding positions.